數位電路實驗結報一

–6-bit 數位加法器–

Group 2: B11303052 蘇盈翰 (Right), B11702091 張容爾 (Middle), B10901144 王俊皓 (Left)

April 8, 2026

1 實驗理論與原理

1.1 半加器與全加器 (Half Adder & Full Adder)

加法器為數位邏輯電路中執行二進位加法運算之核心組件。最基本的**半加器**負責兩個單一位元 A 與 B 的加法，其輸出包含總和 (S) 與進位 (C_{out})，布林邏輯表示式為：

$$\mathrm{S} = \mathrm{A} \oplus \mathrm{B} \tag{1}$$

$$\mathrm{C_{out}} = \mathrm{A} \cdot \mathrm{B} \tag{2}$$

為了處理多位元的加法，必須考慮來自較低位元的進位。因此引入了**全加器**，它包含三個輸入端：兩個加數 (A, B) 以及一個進位輸入 (C_{in})。全加器的輸出邏輯表示式為：

$$\mathrm{S} = \mathrm{A} \oplus \mathrm{B} \oplus \mathrm{C_{in}} \tag{3}$$

$$\mathrm{C_{out}} = (\mathrm{A} \cdot \mathrm{B}) + (\mathrm{C_{in}} \cdot (\mathrm{A} \oplus \mathrm{B})) \tag{4}$$

1.2 邏輯閘實現與笛摩根定理

在實體電路佈線中，我們使用 IC 7486 (XOR 閘) 與 IC 7400 (NAND 閘) 來建構上述邏輯。總和 S 可直接透過兩個 XOR 閘串接完成。

對於進位 C_{out} 的部分，原本的設計需要 AND 閘與 OR 閘。為了僅使用 NAND 閘 (IC 7400) 來完成進位電路，我們可套用**笛摩根定理 (De Morgan's Laws)** 將原本的 AND-OR 形式轉換為 NAND-NAND 形式：

$$\mathrm{C_{out}} = \overline{\overline{(\mathrm{A} \cdot \mathrm{B})} \cdot \overline{(\mathrm{C_{in}} \cdot (\mathrm{A} \oplus \mathrm{B}))}} \tag{5}$$

透過此轉換，即可大幅簡化所需的 IC 種類，完全依靠 7486 與 7400 實現 1-bit 全加器。

1.3 漣波進位加法器 (Ripple Carry Adder)

要實現多位元（如本實驗之 6-bit）的加法運算，可將多個全加器串聯組成「漣波進位加法器」。其核心原理是將第 i 個位元的進位輸出 ($\mathrm{C_{out,i}}$) 直接連接至第 $i+1$ 個位元的進位輸入 ($\mathrm{C_{in,i+1}}$)。

本次實驗中，我們將兩個現成的 3-bit 加法器模組進行擴充，把低位元 (Lower 3-bits) 加法器的最高位進位輸出，連接至高位元 (Upper 3-bits) 加法器的最低位進位輸入 ($\mathrm{C_{in}}$)，藉由進位訊號的傳遞 (Carry Propagation)，成功構建出完整的 6-bit 數位加法器。

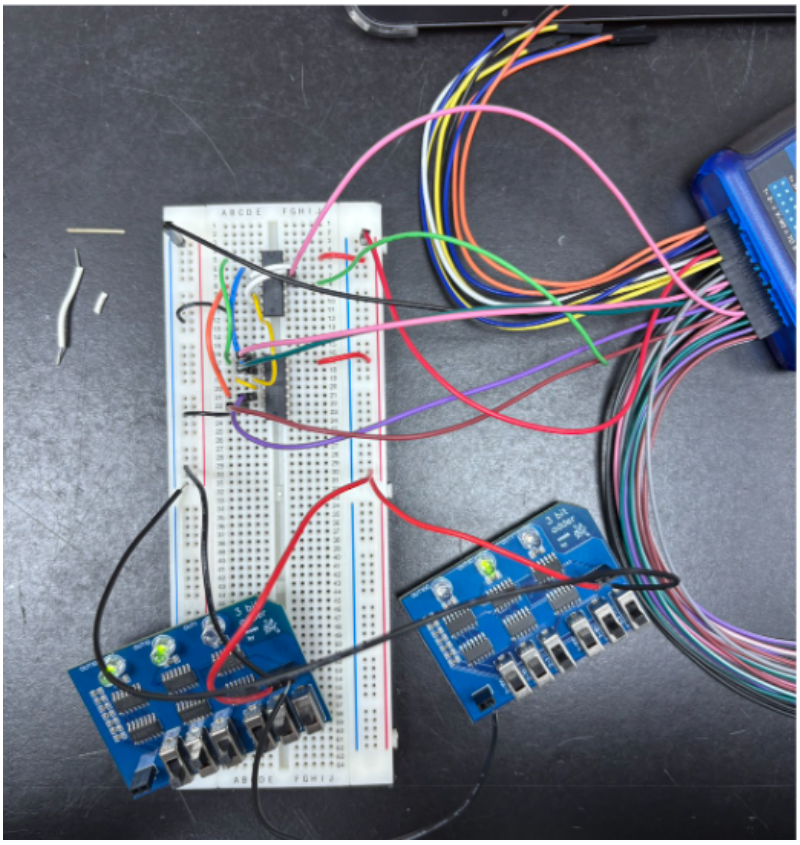
2 實驗架設與步驟

2.1 1-bit 數位加法器實作 (實驗一)

1. **硬體接線：**

   - 準備 IC 7486 (XOR) 與 IC 7400 (NAND)，將其固定於麵包板上。分別將兩顆 IC 的第 14 腳位連接至 ADALM2000 提供的 +5V 電源 (VCC)，第 7 腳位連接至接地 (GND)。

- 依據理論化簡後之電路圖進行佈線：利用 7486 負責處理總和 (S) 與部分進位邏輯，並將其輸出結果接入 7400 進行 NAND 邏輯運算，以產生最終的進位輸出 (C_{out})。
- 將 ADALM2000 的數位腳位 (DIO) 連接至電路對應節點：DIO 0 接至輸入 A、DIO 1 接至輸入 B、DIO 2 接至輸入 C_{in}；量測端的部分，將 DIO 3 接至總和輸出 S、DIO 4 接至進位輸出 C_{out}。



2. **軟體設定 (Scopy)：**
   - **Pattern Generator (訊號產生器)：**開啟 Scopy 軟體，將 DIO 0、1、2 設定為 Binary Counter 模式，並設定為三個等比頻率（例如 A 設為 4 kHz、B 設為 2 kHz、C_{in} 設為 1 kHz）。此設定可產生所有 8 種 0 與 1 的輸入排列組合。
   - **Logic Analyzer (邏輯分析儀)：**開啟 DIO 0 至 DIO 4 的波形顯示。將取樣率 (Sample Rate) 設定為 5 Msps，取樣數量 (Nr of samples) 設定為 20k Samples。
   - 擷取波形，觀察輸出腳位與輸入腳位之電位變化，並比對全加器真值表以驗證硬體邏輯。

2.2 多位元加法器串聯測試 (實驗二)

1. **3-bit 加法器模組測試：**取得助教事先製作的 3-bit 加法器電路板，透過板上的指撥開關給定不同的輸入組合，並觀察 LED 燈亮的狀態來驗證 3-bit 的加法結果（例如測試 $3 + 5 = 8$，即二進位之 $011_2 + 101_2 = 1000_2$）。

2. **6-bit 加法器擴充：**將兩塊 3-bit 加法器模組進行串聯。把處理低位元 (Lower 3-bits) 模組的進位輸出端 (C_{out})，利用跳線連接至處理高位元 (Upper 3-bits) 模組的進位輸入端 (C_{in})。

3. 透過 ADALM 再次進行輸入測試，確認進位訊號能順利跨模組傳遞，完成並驗證完整的 6-bit 加法運算邏輯。

3 實驗結果與分析

3.1 1-bit 數位全加器邏輯驗證

透過 ADALM2000 擷取訊號，我們記錄了 1-bit 全加器在 8 種不同輸入組合下的輸出結果（如表 1 所示）。分析該測量數據可得出以下結論：

狀態	1st	2nd	3rd	4th	5th	6th	7th	8th
A	1	0	1	0	1	0	1	0
B	0	1	1	0	0	1	1	0
Cin	1	1	1	0	0	0	0	1
S	0	0	1	0	1	1	0	1
Cout	1	1	1	0	0	0	1	0

Table 1: 1-bit 全加器輸入與輸出之實測真值表

- **總和 (S) 邏輯分析：**當輸入端 A, B, C_{in} 中有「奇數」個 1 處於高電位時（即狀態 3rd, 5th, 6th, 8th），總和輸出 S 皆為 1。這完美吻合 XOR 閘 ($A \oplus B \oplus C_{in}$) 的奇偶校驗 (Parity Check) 特性。
- **進位 (C_{out}) 邏輯分析：**當輸入端有「兩個或三個」1 處於高電位時（即狀態 1st, 2nd, 3rd, 7th），進位輸出 C_{out} 皆為 1。這證明了我們利用 IC 7400 (NAND 閘) 配合笛摩根定理所建構的進位電路，成功實現了全加器的多數決邏輯 (Majority Logic)。

實測之數位波形電壓準位與理論真值表完全吻合，證實了本實驗的硬體佈線與邏輯化簡皆正確無誤。

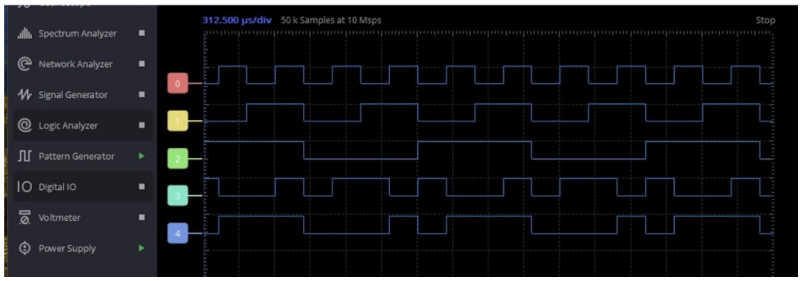


Figure 1: 1-bit 全加器輸入與輸出之實測波形圖

3.2 6-bit 加法器串聯結果與訊號傳遞分析

在進階實驗中，我們成功將兩個 3-bit 加法器模組串接，擴充為完整的 6-bit 加法器。測試過程中，我們輸入了多組十進位加法進行二進位驗證。例如測試 $3 + 5 = 8$ 時（即二進位之 $000011_2 + 000101_2$）：

- **運算過程觀察：**由於低位元（第 1、2 個 bit）的相加產生了進位，我們可以明確觀察到低位元模組的 C_{out} 輸出高電位，並成功觸發高位元模組的 C_{in} 接收端，最終正確點亮對應 1000_2 （即數值 8）的輸出 LED 燈號。

外的硬體邏輯，提前計算出各個位元是否會產生或傳遞進位，從而打破逐級等待的限制，大幅提升數位電路的運算效能。

4 問題討論

4.1 如何使用二進位表示負數？

在數位電路中，最常使用二**補數 (Two's Complement)** 系統來表示有號數（包含正負數）。其轉換規則為：將正數的二進位表示法的所有位元反轉（0 變 1，1 變 0，即求取「一補數」），然後在最低位元 (LSB) 加 1。

在二補數系統中，最高位元 (MSB) 通常被視為**符號位元 (Sign Bit)**：0 代表正數，1 代表負數。採用二補數系統最大的優勢在於，硬體電路在執行減法時，可以直接將其視為「加上一個負數」，因此加法與減法運算可以共用同一組加法器硬體，大幅簡化了電路設計的複雜度。

4.2 如何用數位電路實現乘法、指數運算？

- **乘法運算：**數位電路中的乘法基本原理是「**移位與相加**」**(Shift and Add)**。過程類似於人類計算十進位直式乘法，乘數的每一個位元會與被乘數進行 AND 邏輯運算，產生「部分積 (Partial Product)」。接著，將這些部分積依據權重向左移位，最後透過多組加法器（例如陣列乘法器 Array Multiplier）將所有部分積相加，即可得到最終乘積。

- **指數運算：**指數運算本質上是「**重複的乘法**」。在基礎硬體設計上，可藉由有限狀態機 (FSM) 控制硬體乘法器，進行多次的迴圈相乘。然而，針對複雜的指數函數（如 e^x），商用晶片通常會採用**查表法 (Look-Up Table, LUT)** 直接輸出預先算好的結果，或是利用**泰勒級數 (Taylor Series) 展開**，將指數函數近似為加法與乘法的多項式組合，以換取更高的運算效率。

4.3 進行多位元加法時（如實驗二），可能會遇到哪些問題或限制？商用電路有什麼優化電路的方式嗎？

遇到的問題與限制：進位延遲 (Carry Propagation Delay)
如本次實驗二所串聯的 6-bit 加法器，其架構屬於「漣波進位加法器 (Ripple Carry Adder, RCA)」。此架構的致命缺點是：高位元的全加器必須等待低位元計算完畢並將進位訊號 (C_{out}) 傳遞過來後，才能得出正確的結果。當位元數增加（如現代 CPU 的 32-bit 或 64-bit 加法），訊號經過層層邏輯閘所累積的時間延遲會非常可觀，這會嚴重限制處理器的最高工作時脈 (Clock Speed)。

商用電路的優化方式：預看進位加法器 (Carry Lookahead Adder, CLA)
為了解決 RCA 的延遲瓶頸，商用處理器通常採用 CLA 架構。CLA 放棄了逐級等待進位的作法，而是透過增加額外的組合邏輯電路，利用輸入的 A 與 B 訊號提前計算出「進位產生 (Generate, G)」與「進位傳遞 (Propagate, P)」的狀態。透過這些狀態，電路可以在極短的時間內「同時」算出所有位元的進位訊號，從而打破線性延遲的限制，大幅提升多位元運算的效能。